2025 8th International Conference on Information and Computer Technologies (ICICT) | 979-8-3315-0518-9/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICICT64582.2025.00037

Lightweight Relevance Grader in RAG

Taehee Jeong
San Jose State University
San Jose, CA, USA
taehee.jeong@sjsu.edu

***Abstract*—Retrieval-augmented generation (RAG) addresses limitations of large language models (LLMs) by leveraging a vector database to provide more accurate and up-to-date information. When a user submits a query, RAG executes a vector search to find relevant documents, which are then used to generate a response. However, ensuring the relevance of retrieved documents is a challenge. To address this, a secondary model, known as a relevant grader, is used to verify document relevance. To reduce computational requirements, a lightweight small language model can be used as a relevant grader. This work aims to improve the capability of such a model, achieving a significant increase in precision from 0.1038 to 0.7750 using llama-3.2-1b, outperforming llama-3.1-70b and gpt4o-mini. Our code is available at https://github.com/taeheej/Lightweight-Relevance-Grader-in-RAG.**

Keywords—Retrieval-augmented generation, Large language models, Relevance grader, Fine-tuning, Vector search, Vector database

I. Introduction

Large Language Models (LLMs) have achieved exceptional performance in various Natural Language Processing (NLP) tasks [1]-[3], demonstrating their ability to absorb and retain vast amounts of knowledge. When responding to specific queries, LLMs often provide valuable and informative answers, leveraging the extensive range of information they acquired during their training. However, while their capabilities are impressive, LLMs still have several limitations that hinder their overall effectiveness.

A major limitation of LLMs is the rapid escalation in the number of parameters required to achieve high performance. As the training dataset expands, the model needs to capture increasingly complex patterns, which in turn demands a substantial increase in parameters. This exponential growth not only adds complexity to the model but also creates significant deployment challenges, making it more difficult to implement the model in real-world applications.

Another limitation of LLMs is their inability to effectively incorporate time-sensitive or non-public information. This limitation arises from the fact that LLMs are trained on static datasets that represent a snapshot of the internet at a particular point in time. As a result, these models often lack access to the most recent developments or updates. This can lead to a critical issue: LLMs may generate "fact hallucinations," where they produce responses that are not grounded in factual information. This problem is particularly alarming in applications where accuracy and reliability are crucial, as it can erode trust in the model's outputs and have significant consequences.

A novel approach has recently emerged to tackle these limitations: Retrieval-Augmented Generation (RAG) [4], [5]. The RAG enhances its capabilities by integrating LLMs with external knowledge retrieval. This integration allows the RAG to access and incorporate not only publicly available information but also time-sensitive data and information that is not publicly accessible, thereby expanding its knowledge base.

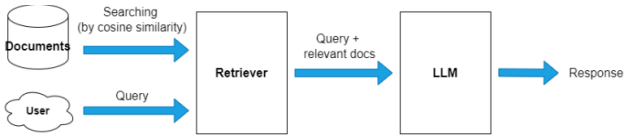

Fig. 1. RAG system

The RAG system works by first accepting an input query. It then uses a neural retriever to search an external knowledge database and retrieve the most relevant documents related to the query. Next, these documents are combined with the original query to create a prompt for the language model. The language model then generates its output based on the information from the retrieved documents, resulting in a comprehensive and informed response to the query. The work flow of the RAG system is illustrated in Fig. 1.

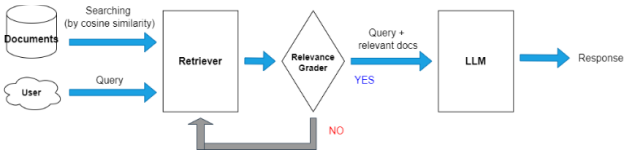

Fig. 2. RAG system with a relevance grader

The RAG system differs from generative-only models in its ability to utilize time-sensitive information and non-public documents, such as internal company documents, to reduce the risk of hallucinations. A key component of RAG is its document retrieval mechanism, which involves comparing a query vector to document vectors in a database using cosine similarity. The top matches are then selected, but this process may still yield some irrelevant documents. To refine the results, RAG employs a re-ranking process, where a secondary LLM acts as a relevance grader. This LLM assesses the retrieved documents to determine their suitability for answering the user's question, ensuring that the final response is relevant and accurate. The work flow of the RAG system with a relevance grader is illustrated in Fig. 2.

2769-4542/25/$31.00 ©2025 IEEE
DOI 10.1109/ICICT64582.2025.00037

Authorized licensed use limited to: Munster Technological University Ireland. Downloaded on January 28,2026 at 22:00:27 UTC from IEEE Xplore. Restrictions apply.

Integrating an additional LLM into the RAG pipeline poses significant memory and computational challenges. To overcome these limitations, we propose using a fine-tuned, small language model as the relevance grader. A key challenge is that a language model's capability is often tied to its number of parameters [19], making it essential to explore methods that can achieve sufficient capability with a relatively small number of parameters. Given that our baseline model has significantly fewer parameters than widely-used large language models, we anticipated potential performance issues with relevance grading. To mitigate this, we added a binary classification head to the model's final layer, which is suitable for the binary output of the relevance grader. We then fine-tuned the model under various hyper-parameter configurations, incorporating Contrastive loss to further optimize its performance.

The primary contributions of this paper are as follows:

- Model Enhancement: A fine-tuned small language model (lama-3.2-1b) is used for relevance grading in RAG systems, improving precision from 0.1038 to 0.7756.
- Efficiency and Speed: The compact model minimizes memory and computational requirements, enabling deployment in resource-constrained environments and accelerating the retrieval process.
- Dataset and Evaluation: A large dataset of 45,000 query-document pairs is created for evaluating and fine-tuning the relevance grading process, supporting the development of more accurate RAG systems.

Overall, the paper contributes to the advancement of RAG systems by offering a practical and efficient solution for relevance grading, which enhances the accuracy and performance of information retrieval in the presence of limited computational resources.

II. Related work

To identify relevant documents from a knowledge database, searching algorithms are employed in RAG systems. Traditional search algorithms rank documents by the frequency of query terms within them. Among the widely used algorithms are Term Frequency-Inverse Document Frequency (TF-IDF) and Best Matching 25 (BM25) [6]. However, these approaches primarily depend on lexical matching, which can limit their ability to effectively grasp the semantic meaning or context of a query.

Unlike traditional search algorithms that rely on exact keyword matches, vector search utilizes vector embeddings to capture the semantics of data, enabling a meaning-based search approach. In this method, both the query and the document are independently transformed into embedding vectors using a semantic encoder. Vector search then assesses the similarity between the query vector and potential vectors stored in a database. This technique allows unstructured data such as images, text, and audio to be represented as vectors in a high-dimensional space, facilitating the efficient identification and retrieval of vectors that closely align with the query vector.

Distance metrics, like Euclidean distance and cosine similarity, are frequently employed to evaluate the similarity between vectors. The Euclidean distance between two embedding vectors, $\boldsymbol{v(s_1)}$ and $\boldsymbol{v(s_2)}$, each with n dimensions representing sentence 1 and sentence 2, is defined as follows:

$$d(s_1, s_2) = \|\boldsymbol{v(s_1)} - \boldsymbol{v(s_2)}\|_2 = \sqrt{\sum_{i=1}^{n-1}(v_i^1 - v_i^2)^2} \quad (1)$$

Cosine similarity between two vectors $\boldsymbol{v(s_1)}$ and $\boldsymbol{v(s_2)}$ is also defined as follows.

$$sim(s_1, s_2) = \frac{\boldsymbol{v(s_1)} \cdot \boldsymbol{v(s_2)}}{\|\boldsymbol{v(s_1)}\|_2 \|\boldsymbol{v(s_2)}\|_2} = \frac{\sum_{i=1}^{n-1}(v_i^1 \cdot v_i^2)}{\sqrt{\sum_{i=1}^{n-1}(v_i^1)^2}\sqrt{\sum_{i=1}^{n-1}(v_i^2)^2}} \quad (2)$$

Hybrid search, a recently introduced method, integrates keyword-based search with vector-based search to capitalize on the strengths of both techniques. This combination has the potential to yield more precise and relevant search results. In a hybrid search system, keyword-based and vector-based searches are performed separately, and their results are subsequently merged. Despite its promise, one of the challenges lies in ranking these results and assigning appropriate weights to effectively combine them.

The ever-increasing volume of accessible information resources has created a significant demand for effective methods of similarity searching. Vector search algorithms are specifically developed to efficiently identify the vectors most similar to a given query vector within a high-dimensional space. Among the widely used vector searching algorithms are K-Nearest Neighbors (KNN) and Approximate Nearest Neighbor (ANN).

KNN [7], often referred to as the Brute Force algorithm, identifies the K nearest vectors to a query vector by measuring the distance—typically the Euclidean distance—between the query and every other vector in the dataset. While it ensures the precise identification of the nearest neighbors, it can be computationally demanding for large datasets.

ANN [8] algorithms permit a slight error margin, providing points that are nearly the closest rather than exactly the nearest. While this approach sacrifices some precision, it offers a substantial increase in speed over exact nearest neighbor search methods. Among the various ANN algorithms, Hierarchical Navigable Small World (HNSW) algorithm is the most widely used.

HNSW [9] creates a hierarchical graph structure, where each node corresponds to a data point and edges link nearby points in the dataset. This graph is composed of multiple layers, each representing a different level of detail or resolution. These layers are arranged hierarchically, with broader, coarser layers at the top and more detailed, finer layers at the bottom. The algorithm's main advantage lies in its ability to efficiently narrow down the search space by navigating through these layers to find the most likely candidates for nearest neighbors. This process begins at the top layer and progressively moves down to the lower layers, using the edges to steer the search towards the most similar data points. As a result, HNSW effectively balances the trade-off between search speed and accuracy.

Authorized licensed use limited to: Munster Technological University Ireland. Downloaded on January 28,2026 at 22:00:27 UTC from IEEE Xplore. Restrictions apply.

Within RAG pipelines, re-ranking [20]-[22] plays a crucial role in refining initial search results to better align with user intent and context. By doing so, it enhances user satisfaction by delivering more precise and contextually relevant outcomes. This, in turn, leads to increased conversion rates and improved engagement metrics. Ultimately, re-ranking enables Large Language Models (LLMs) to leverage the most relevant and high-quality information available, resulting in more accurate and effective results.

Cross-encoders play a crucial role in re-ranking processes within RAG pipelines. Their functionality involves taking a concatenated query and document as input, generating a relevance score as output. Although cross-encoders excel at capturing the nuanced interactions between queries and documents, their computational requirements are substantial. This is largely due to the fact that LLMs are often utilized as cross-encoders, which demands significant memory and computational resources.

The development of lightweight language models as cross encoders seeks to strike a balance between accuracy and efficiency. With their faster processing speeds and smaller memory requirements, these models are well-suited for real time applications. However, they often struggle to match the accuracy and contextual relevance of their larger counterparts. To address this limitation, our research focuses on creating a fine-tuned, lightweight language model that functions as a relevant grader. The goal of this model is to provide search results that are comparable in accuracy and relevance to those produced by larger, more complex language models.

III. Dataset

A. Data Preparation

To evaluate the accuracy of search results, we conducted a comprehensive study using 45,000 pairs of user queries and corresponding recent news articles. Our approach involved two main steps. First, we created a vector database of news articles by collecting articles daily from multiple news sources [24] and embedding them using the bge-small-en-v1.5 [10] semantic encoding model. This resulted in 384-dimensional vectors, which were then stored on Amazon DynamoDB. In the second step, we developed a set of 20 query questions across eight distinct fields: Pharmacy, Venture Capital, Information Technology (IT), Legal, Banking, Healthcare, Automotive, and Residential Construction. These queries were carefully crafted to cover various categories, including R&D, Technology, Regulations, Market, Manufacturing, Hiring, Sustainability, Business-to-Business (B2B), Security, Industry, Leadership, Economy, and Finance. For example, one such query was *"How will the expanding specialized drug market impact pharmaceutical R&D strategy and manufacturing capabilities?"*. In total, we had 160 unique queries.

To identify relevant news articles for each query, we embedded the query sentences using the same bge-small-en v1.5 model and calculated the cosine similarity between the query vector and the news article vectors. We utilized the HNSW algorithm for efficient vector searching, which enabled us to find the top five most similar vectors for each query. This process was repeated daily over a 90-day period for all 160 queries, resulting in the collection of 45,000 query-article pairs.

B. Relevant Grading

To guarantee that user intent and context are aligned, we combined the query with pertinent documents as input and assessed their relevance using Llama-3.1-405B-Instruct [11]. This model is the largest and most advanced openly accessible foundation model [23]. We utilized the following system prompt, incorporating chain-of-thought phrases: *"Please analyze the contents of DOCUMENTS and determine whether it is relevant in answering the QUESTION"*

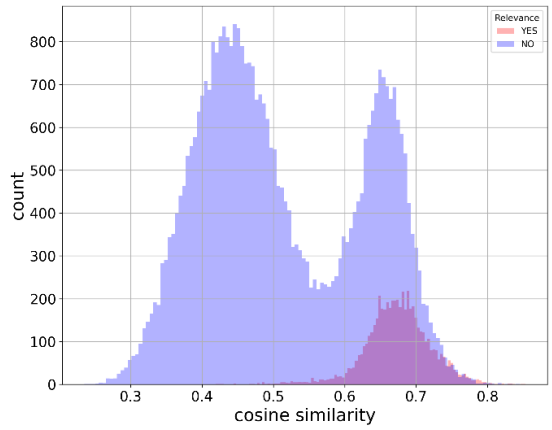

Fig. 3. Distribution of cosine similarity with relevant grading

Fig. 3 illustrates the distribution of cosine similarity along with the evaluation outcomes for relevant grading. It reveals that merely 12.3% of the cases are approved by the relevant grader, highlighting the essential function of the relevant grader within the RAG pipeline. Additionally, the distribution displays a bimodal pattern. The second peak in this distribution corresponds to relevance, while the first peak appears to be misaligned with the relevant search. This misalignment could be attributed to the HNSW's nature as an approximate search method, which may compromise accuracy, or to an imprecise embedding model.

We evaluated its relevance outcomes against other LLMs, including gpt4o-mini [12], Llama-3.1-70B-Instruct [13], Llama-3.1-8B-Instruct [14], Llama-3.2-3B-Instruct [15], and Llama-3.2-1B-Instruct [16]. The relevance grading results from Llama-3.1-405B-Instruct were used as the Ground-True, and we calculated Accuracy, Precision, Recall, and F1-score based on the confusion matrix according to Table I, Eq 3, Eq 4, Eq 5 and Eq 6. The findings are presented in Table II.

TABLE I. Confusion Matrix

		Predicted label	
		1	***0***
True label	1	true positive (tp)	false negative (fn)
	0	false positive (fp)	true negative (tn)

Authorized licensed use limited to: Munster Technological University Ireland. Downloaded on January 28,2026 at 22:00:27 UTC from IEEE Xplore. Restrictions apply.

$$Accuracy = \frac{\text{tp+tn}}{\text{tp+fp+fn+tn}} \quad (3)$$

$$Precision = \frac{\text{tp}}{\text{tp+fp}} \quad (4)$$

$$Recall = \frac{\text{tp}}{\text{tp+fn}} \quad (5)$$

$$F_1 = \frac{2}{\frac{1}{Precision} + \frac{1}{Recall}} \quad (6)$$

TABLE II. MODEL COMPARISON FOR RELEVANCE GRADING

Model	*Accuracy*	*Precision*	*Recall*	*F1-score*
gpt4o-mini	0.9302	0.7170	0.7118	0.7144
llama3.1-70b	0.9582	0.8341	0.8225	0.8283
llama3.1-8b	0.8815	0.5116	0.7607	0.6118
llama3.2-3b	0.7793	0.3385	0.8372	0.4820
llama3.2-1b	0.2367	0.1312	0.9288	0.2299

The dataset is imbalanced, comprising a majority of negative labeled data. This imbalance can result in a high false positive rate in a model's predictions. To effectively evaluate the model's performance in this context, Precision is a particularly useful metric, as it helps assess the accuracy of the model's positive predictions. As anticipated, a model with a large number of parameters, like Llama-3.1-70B, achieves the highest Precision score of 0.8341. In contrast, a model with fewer parameters, such as Llama-3.2-1B, has the lowest Precision score of 0.1312, in line with scale's law [19]. Despite Llama-3.2-1B having the poorest Precision among the other large language models, it is suitable for efficient deployment in RAG systems due to its lightweight design, which requires less memory and computing power. Our objective in this work is to fine-tune Llama-3.2-1B to enhance its Precision, enabling it to function effectively as a relevance grader.

IV. TASK-SPECIFIC FINE-TUNING

Fine-tuning a LLM on specialized data allows it to leverage its extensive pre-learned knowledge and adapt to a specific task. By modifying its parameters through fine-tuning, the model can better align with the demands of the task, resulting in improved performance and applicability within that domain. This approach is particularly effective when we want to optimize the model's performance for a single, well-defined task, ensuring that the model excels in generating task-specific content with precision and accuracy.

Our work began with the Llama-3.2-1B model as our foundation. We aimed to fine-tune this baseline model to perform as a Relevant Grader, a task that requires assessing the relevance between a user's query and a set of pertinent documents. Specifically, the model would take a user's query and a related document as input and output a determination of whether the query and the document are relevant to each other. To avoid overfitting, we divided the dataset of 45,000 user query and document pairs into 80% for training and 20% for testing. The training and testing datasets preserve the same proportion of positive to negative labels.

The fine-tuning process involves adjusting the parameters of a LLM to better suit a specific task. The degree of modification can vary greatly depending on the task's requirements. Broadly speaking, fine-tuning can be approached in two distinct ways: full fine-tuning and transfer learning.

A. *Full fine-tuning*

Full fine-tuning involves a comprehensive adjustment of a model, where all parameters of its layers are modified using data that is specifically tailored to a particular task. In our case, we fine-tuned every layer of Llama-3.2-1B-Instruct using a training dataset consisting of 36,000 pairs of user query and document.

TABLE III. MODEL COMPARISON ON TEST DATASET

Model	*Accuracy*	*Precision*	*Recall*	*F1-score*
gpt4o-mini	0.9280	0.7042	0.7131	0.7086
llama3.1-70b	0.9573	0.8239	0.8299	0.8269
llama3.1-8b	0.8848	0.5208	0.7710	0.6217
llama3.2-3b	0.7722	0.3309	0.8371	0.4744
llama3.2-1b	0.2114	0.1038	0.9128	0.1863
fft[a]-llama3.2-1b	0.7439	0.1655	0.2688	0.2049

[a] fft: full fine-tuned

We utilized the AdamW optimizer [17] with a cosine learning rate schedule. The schedule started with an initial learning rate of 2e-5 and gradually decreased to a final learning rate that was 10% of the peak rate. Cross-entropy was used as the loss function. Since the training dataset was skewed, with a predominance of negative labels, we implemented both oversampling and under-sampling techniques to achieve a more balanced distribution of positive and negative labels, thereby mitigating the impact of class imbalance on our model's performance.

After fine-tuning the model, we evaluated its performance on the test dataset by measuring Accuracy, Precision, Recall, and F1-score, and compared these metrics with other large language models, as shown in Table III. Our fully fine-tuned (fft) Llama-3.2-1B demonstrated an improvement in Precision, increasing from 0.1038 to 0.1655, although it still lags behind the Precision of Llama-3.2-70B.

B. *Transfer learning with Classification head*

Our study employed transfer learning, a technique that harnesses knowledge gained from one task or pre-existing knowledge obtained through pre-training on a large dataset to enhance performance on a specific task. To implement this approach, we leveraged a pre-trained Llama model and attached a classification head, a specialized layer designed for classification tasks, to its end. The classification head plays a crucial role in predicting the final label by processing the model's output. Specifically, it takes the hidden state with a dimension of 2048 and converts it into a logit with a dimension of 2, corresponding to the number of labels. The logit then

Authorized licensed use limited to: Munster Technological University Ireland. Downloaded on January 28,2026 at 22:00:27 UTC from IEEE Xplore. Restrictions apply.

undergoes softmax and argmax processing to yield the final label. A significant benefit of this transfer learning approach is the substantial reduction in computational operations required during training. By utilizing a pre-trained model, we avoided the need to train a large model with 1.236 billion parameters, instead training only a single classification layer with 4096 parameters, resulting in considerable computational savings.

There are two distinct types of loss functions used in classification: cross-entropy loss and contrastive loss [18]. Cross-entropy loss evaluates the class probabilities generated by a model for each sample individually, without imposing any constraints on the model's internal representation of the input. In contrast, contrastive loss operates on pairs of embeddings produced by the model, along with a Boolean label indicating whether the two vectors are similar or not. The key difference between these two loss functions lies in how they handle similar and dissimilar pairs during training. Contrastive loss penalizes similar pairs if they are too far apart, based on Euclidean distance, while dissimilar pairs are penalized if they are too close. To achieve this, contrastive loss introduces a concept called a "margin," which is the minimum distance that dissimilar points must maintain. If the distance between two vectors is less than the margin, it suggests that the pair is likely relevant, whereas a distance greater than the margin indicates that the vectors probably represent different pairs.

We employed both Cross-entropy and Contrastive loss as our loss functions. The Contrastive loss is defined as following.

$$Loss = (1 - \mathrm{Y})\frac{1}{2}{D_w}^2 + Y\frac{1}{2}[\max(0, m - D_w)]^2 \qquad (7)$$

where D_w is Euclidean distance between the two input vectors, m> 0 is a margin, and Y is a binary label assigned to this pair.

In this study, we represented both the model's output and the actual label as vectors. We then calculated the Euclidean distance between these two vectors, using a value of 1 for the parameter m. Table IV presents a comparison of the fine-tuned model's performance using Cross-entropy loss versus Contrastive loss. The model's Precision with Contrastive loss surpasses that with Cross-entropy loss and is comparable to the results achieved with full fine-tuning IV-A.

TABLE IV. MODEL COMPARISON BY TRANSFER LEARNING ON TEST DATASET

Model	*Accuracy*	*Precision*	*Recall*	*F1-score*
llama3.2-1b	0.2114	0.1038	0.9128	0.1863
tl[a]-llama3.2-1b (Cross-entropy loss)	0.5730	0.1411	0.4869	0.2187
tl[a]-llama3.2-1b (Contrastive loss)	0.7356	0.1657	0.2860	0.2098

[a] tl: transfer learning

C. Full fine-tuning with Classification head

In the previous section IV-B, we explored a method where a pre-trained LLM was used as a fixed feature extractor, with a classification head appended to its end for a specific classification task. The pre-trained LLM provided comprehensive representations, which were then tailored to the task at hand by training the final layers on a relevance grading dataset, while keeping the rest of the model unchanged. This approach allowed for efficient fine-tuning of the LLM. However, despite observing an improvement in precision, the results did not fully meet our expectations. To further improve performance, we also experimented with fully fine-tuning the model, including the addition of a classification head, which involved training the entire model on task-specific data. Unlike the previous approach, which only modified the final layers, full fine-tuning adjusted all model layers during training, providing an alternative to traditional transfer learning methods.

TABLE V. MODEL COMPARISON ON TEST DATASET

Model	*Accuracy*	*Precision*	*Recall*	*F1-score*
gpt4o-mini	0.9280	0.7042	0.7131	0.7086
llama3.1-70b	0.9573	0.8239	0.8299	0.8269
llama3.1-8b	0.8848	0.5208	0.7710	0.6217
llama3.2-3b	0.7722	0.3309	0.8371	0.4744
llama3.2-1b	0.2114	0.1038	0.9128	0.1863
fft[a]-llama3.2-1b w classification head (Cross-entropy loss)	0.9353	0.7750	0.6670	0.7170
fft[a]-llama3.2-1b w classification head (Contrastive loss)	0.9291	0.7256	0.6796	0.7018

[a] fft: full fine-tuned

In Table V, we compared our results with other pre-trained LLMs. Our fully fine-tuned llama-3.2-1b with a classification head achieved a Precision of 0.7750, which is significantly higher than that of llama3.1-8b and gpt4o-mini, but slightly below that of llama3.1-70b.

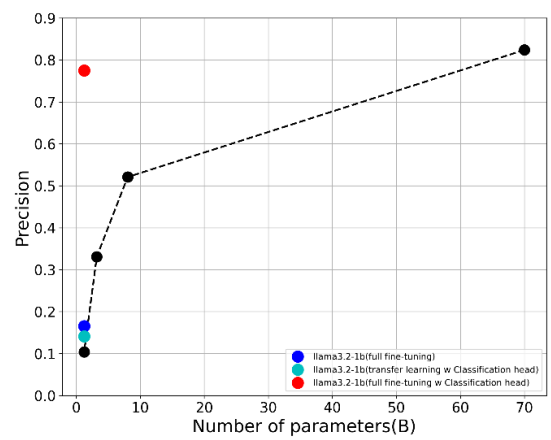

Fig. 4. Precision of relevance grading on test dataset

The relationship between model complexity and precision is illustrated in Fig. 4, which shows that models with a larger number of parameters tend to achieve higher precision on the test dataset. To further evaluate our fine-tuned models, we used cross-entropy loss as a comparison metric. Our full fine-tuned llama-3.2-1b model with a classification head, demonstrated

Authorized licensed use limited to: Munster Technological University Ireland. Downloaded on January 28,2026 at 22:00:27 UTC from IEEE Xplore. Restrictions apply.

particularly impressive results. Notably, it exceeded the typical performance expectations outlined by the scale's law, suggesting that our approach can lead to exceptional outcomes.

V. Conclusion

Recently, RAG has emerged as a novel approach to address the limitations of LLMs. One major issue with LLMs is their propensity to provide outdated information and generate hallucinations, leading to incorrect or misleading responses. To combat this, RAG leverages similarity search to retrieve relevant data from a vector database that encompasses a wide range of media, including images, audio, video, and text. The core of RAG relies on cosine similarity to search for related documents, which is crucial for information retrieval. However, the documents retrieved through cosine similarity often fail to fully address the query, necessitating a re-ranking process to assess their relevance.

To refine the search results, a re-ranking process is employed, which typically involves an additional LLM as a relevance grader. However, this approach is resource-intensive, requiring significant memory and computational power within RAG pipelines. A potential solution is to utilize a smaller language model as a relevance grader, but these models often underperform. To overcome this challenge, we have employed a fine-tuned small language model specifically for the relevance grading task, which aims to strike a balance between efficiency and accuracy.

To develop an effective relevance grader for a Retrieval-Augmented Generation (RAG) system, we created a dataset of 45,000 query-question and document pairs. We started with the lama-3.2-1b model as our foundation and enhanced it by adding a classification head layer. Through full fine-tuning using a combination of Cross-entropy loss and Contrastive loss, we significantly improved the model's Precision from 0.1038 to 0.7756, making it suitable for relevance grading.

The use of a compact language model like lama-3.2-1b offers several advantages in a RAG system. Firstly, it reduces the demand for memory bandwidth and computational power, making it more feasible to implement RAG systems in resource-constrained environments. Additionally, the smaller model size enables faster computation, accelerating the retrieval process. Overall, our approach enables the efficient and effective implementation of RAG systems, striking a balance between performance and resource requirements.

References

[1] OpenAI, “GPT-4 Technical Report,“ arXiv:2303.08774, 2023.

[2] S. Zhang, L. Dong, X. Li, S. Zhang, X. Sun, S. Wang, J. Li, R. Hu, T. Zhang, F. Wu, and G. Wang, “Instruction Tuning for Large Language Models: A Surveyy,“ arXiv:2308.10792, 2024.

[3] S. Minaee, T. Mikolov, N. Nikzad, M. Chenaghlu, R. Socher, X. Amatriain, and J. Gao, “Large Language Models: A Survey,“ arXiv:2402.06196, 2024.

[4] P. Lewisyz, E. Perez, A. Piktusy, F. Petroniy, V. Karpukhiny, N. Goyaly,H. K¨uttlery, M. Lewisy, W. Yihy, T. Rockt¨aschelyz, S. Riedelyz, D.Kiela, “Retrieval-Augmented Generation for Knowledge-Intensive NLPTasks,“ arXiv:2005.11401, 2021.

[5] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Suna, M. Wang,and H. Wang, “Retrieval-Augmented Generation for Large LanguageModels: A Survey,” arXiv:2312.10997, 2024.

[6] S. Robertson and H. Zaragoza, “The Probabilistic Relevance Framework:BM25 and Beyond,“ , Foundations and Trends in Information Retrieval,Vol. 3, No. 4, pp333–389, 2009.

[7] P. N. Yianilos, “Data structures and algorithms for nearest neighbor search in general metric spaces,“ SODA ’93: Proceedings of the fourth annual ACM-SIAM symposium on Discrete algorithms, vol.93, no. 194, pp.311-321, 1993.

[8] A. Andoni, P. Indyk, and I. Razenshteyn, “Approximate Nearest Neighbor Search in High Dimensions,“ , arXiv:1806.09823, 2018.

[9] Y. A. Malkov and D. A. Yashunin, “Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs,“, arXiv:1603.09320, 2018.

[10] https://huggingface.co/BAAI/bge-small-en-v1.5

[11] https://huggingface.co/meta-llama/Llama-3.1-405B-Instruct

[12] https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence/

[13] https://huggingface.co/meta-llama/Llama-3.1-70B-Instruct

[14] https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct

[15] https://huggingface.co/meta-llama/Llama-3.2-3B-Instruct

[16] https://huggingface.co/meta-llama/Llama-3.2-1B-Instruct

[17] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,“, ICIR 2019, arXiv:1711.05101.

[18] R. Hadsell, S. Chopra, and Y. LeCun, “Dimensionality Reduction by Learning an Invariant Mapping,“, CVPR 2006.

[19] G. Sastry, L. Heim, H. Belfield, M. Anderljung, M. Brundage, J. Hazell, C. O’Keefe, G. K. Hadfield, R. Ngo, K. Pilz, G. Gor, E. Bluemke, S. Shoker, J. Egan, R. F. Trager, S. Avin, A. Weller, Y. Bengio, and D. Coyle, “Computing Power and the Governance of Artificial Intelligence,“, arXiv:2402.08797, 2024.

[20] J. Dong, B. Fatemi, B. Perozzi, L. F. Yang, and A. Tsitsulin, “Don’t Forget to Connect! Improving RAG with Graph-based Reranking,“, arXiv:2405.18414.

[21] Y. Yu, W. Ping, Z. Liu, B. Wang, J. You, C. Zhang, M. Shoeybi, and B. Catanzaro, “RankRAG: Unifying Context Ranking with Retrieval-Augmented Generation in LLMs,“, ICIR 2019, arXiv:2407.02485

[22] https://www.pinecone.io/learn/series/rag/rerankers/

[23] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozi`ere, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample, “LLaMA: Open and Efficient Foundation Language Models,“, arXiv:2302.13971.

[24] https://www.gnomi.com/

Authorized licensed use limited to: Munster Technological University Ireland. Downloaded on January 28,2026 at 22:00:27 UTC from IEEE Xplore. Restrictions apply.